

PLAN DE IMPLEMENTACION

Entrega 0.1 - Architectural Baseline

Documento: Plan detallado de implementacion

Version: 1.0

Fecha: 5 de septiembre de 2026

Branch: feat/architectural-baseline

Referencia: hoja_ruta_universal_business_core.pdf

Universal Business Core - Monolito modular, dominio agnostico, multi-tenant

Espera aprobacion antes de proceder a la implementacion.

Documento: Plan detallado de implementación Versión: 1.0 Fecha: 5 de septiembre de 2026 Branch: [feat/architectural-baseline] Documento de autoridad: [hoja_ruta_universal_business_core.pdf]

A. Evaluación actual del repositorio

Estado actual: El repositorio está prácticamente vacío.

Elemento	Estado
Código Python	? Inexistente
[pyproject.toml]	? Inexistente
Estructura de paquetes	? Inexistente
Tests	? Inexistentes
Documentación arquitectónica	? Inexistente (ARCHITECTURE.md, ADRs)
Configuración CI	? Inexistente
[.gitignore]	? Existente, razonablemente completo
Branch	? [feat/architectural-baseline] activo

Activos disponibles:

- [.gitignore] - Listo para usar (venv, cachés, logs, .env, DBs sqlite).
- [hoja_ruta_universal_business_core.pdf] - Especificación de autoridad.

B. Árbol de directorios propuesto

```

app_pedidos_pyme/
??? .gitignore                                # [ya existe]
??? hoja_ruta_universal_business_core.pdf      # [ya existe]
??? pyproject.toml                            # <- NUEVO: metadata, deps, pytest/ruff/mypy config
??? .pre-commit-config.yaml                  # <- NUEVO: hooks opcionales
?
??? docs/
?   ??? ARCHITECTURE.md                      # <- NUEVO: arquitectura al detalle
?   ??? adr/
?       ??? ADR-001-modular-monolith-vs-microservices.md
?       ??? ADR-002-multi-tenancy-strategy.md
?       ??? ADR-003-catalog-item-model.md
?       ??? ADR-004-domain-events-and-outbox.md
?       ??? ADR-005-vertical-extension-model.md
?
??? src/
?   ??? universal_business/
?       ??? __init__.py
?       ?
?       ??? domain/                          # <- PURO, sin imports de infra/api
?           ?   ??? __init__.py
?           ?   ?
?           ?   ??? shared/                  # <- Value objects y primitivas transversales
?               ?   ?   ??? __init__.py
?               ?   ?   ??? value_objects/
?               ?   ?   ?   ??? __init__.py
?               ?   ?   ?   ??? ids.py        # TenantId, BusinessId, LocationId, CustomerId
?               ?   ?   ?   ??? money.py      # Money, Currency (Decimal + ISO-4217)
?               ?   ?   ?   ??? temporal.py   # DateRange, TimeRange, timezone-aware helpers
?               ?   ?   ?   ??? status.py     # LifecycleStatus, StatusTransition
?               ?   ?   ??? entities.py       # BaseEntity + AggregateRoot (genéricos)
?               ?   ?   ??? errors.py         # DomainError, InvariantViolation, etc.
?               ?   ?   ??? events.py        # DomainEvent base + event metadata
?               ?   ?
?               ?   ??? business/            # <- Módulo: Tenant, Business, Location
?                   ?   ?   ??? __init__.py
?                   ?   ?   ??? entities.py   # Tenant, Business, Location, BusinessSettings
?                   ?   ?   ??? value_objects.py # Address, ContactInfo, OperatingHours
?                   ?   ?   ??? ports.py      # ITenant, IBusiness, ILocation repositories
?                   ?   ?
?                   ?   ??? customers/        # <- Módulo: Customers
?                       ?   ?   ??? __init__.py
?                       ?   ?   ??? entities.py # Customer, ContactPoint, Address, Consent
?                       ?   ?   ??? value_objects.py # CustomerPreferences, ConsentType
?                       ?   ?   ??? ports.py   # ICustomerRepository
?                       ?   ?
?                       ?   ??? catalog/       # <- ESQUELETO
?                           ?   ?   ??? __init__.py
?                           ?   ?   ??? entities.py # CatalogItem, Category, Variant placeholders
?                           ?   ?   ??? ports.py   # ICatalogRepository
?                           ?   ?
?                           ?   ??? resources/  # <- ESQUELETO
?                               ?   ?   ??? __init__.py

```

```

?      ?      ?      ??? entities.py          # Resource, ResourceType, Schedule placeholders
?      ?      ?      ??? ports.py
?      ?      ?
?      ?      ??? availability/                # <- ESQUELETO
?      ?      ?      ??? __init__.py
?      ?      ?      ??? entities.py          # AvailabilityRule, Block placeholders
?      ?      ?      ??? ports.py
?      ?      ?
?      ?      ??? reservations/                # <- ESQUELETO
?      ?      ?      ??? __init__.py
?      ?      ?      ??? entities.py          # Reservation, ReservationStatus placeholders
?      ?      ?      ??? ports.py
?      ?      ?
?      ?      ??? orders/                    # <- ESQUELETO
?      ?      ?      ??? __init__.py
?      ?      ?      ??? entities.py          # Order, OrderItem, OrderStatus placeholders
?      ?      ?      ??? ports.py
?      ?      ?
?      ?      ??? fulfillment/                # <- ESQUELETO
?      ?      ??? __init__.py
?      ?      ??? entities.py                # Fulfillment, FulfillmentType placeholders
?      ?      ??? ports.py
?      ?
?      ??? application/                      # <- Casos de uso, comandos, queries
?      ?      ??? __init__.py
?      ?      ??? ports/
?      ?      ??? __init__.py
?      ?      ??? repositories.py            # Re-exporta todos los IRepository
?      ?
?      ??? infrastructure/                   # <- SIN IMPLEMENTAR (solo marcador)
?      ?      ??? __init__.py
?      ?
?      ??? api/                             # <- SIN IMPLEMENTAR (solo marcador)
?      ?      ??? __init__.py
?      ?
?      ??? verticals/                       # <- SIN IMPLEMENTAR (solo marcador)
?      ??? __init__.py
?
??? tests/
    ??? __init__.py
    ??? conftest.py                          # pytest fixtures base (IDs, tz, etc.)
    ?
    ??? architecture/                       # Tests de límites de dependencia
    ?      ??? __init__.py
    ?      ??? test_architecture_boundaries.py
    ?
    ??? unit/
        ??? __init__.py
        ?
        ??? domain/
            ?      ??? __init__.py
            ?      ?
            ?      ??? shared/
            ?      ?      ??? __init__.py

```

```

? ? ??? test_ids.py
? ? ??? test_money.py
? ? ??? test_temporal.py
? ? ??? test_status.py
? ? ??? test_domain_events.py
? ?
? ??? business/
? ? ??? __init__.py
? ? ??? test_tenant.py
? ? ??? test_business.py
? ? ??? test_location.py
? ?
? ??? customers/
?      ??? __init__.py
?      ??? test_customer.py
?
??? application/
      ??? __init__.py

??? .github/
    ??? workflows/
        ??? ci.yml                                     # pytest + ruff + mypy en cada push/PR

```

C. Archivos a crear (resumen por categoría)

Configuración raíz (3 archivos)

1. [pyproject.toml] - metadata, dependencias, configuración de pytest/ruff/mypy
2. [.pre-commit-config.yaml] - hooks opcionales de calidad
3. [.github/workflows/ci.yml] - pipeline de CI

Documentación (6 archivos)

1. [docs/ARCHITECTURE.md]
2. [docs/adr/ADR-001-modular-monolith-vs-microservices.md]
3. [docs/adr/ADR-002-multi-tenancy-strategy.md]
4. [docs/adr/ADR-003-catalog-item-model.md]
5. [docs/adr/ADR-004-domain-events-and-outbox.md]
6. [docs/adr/ADR-005-vertical-extension-model.md]

Paquete fuente `src/universal\_business/` (~44 archivos)

Shared / Domain base:

1. [src/universal_business/__init__.py]
2. [domain/__init__.py]
3. [domain/shared/__init__.py]

4. [domain/shared/value_objects/__init__.py]
5. [domain/shared/value_objects/ids.py]
6. [domain/shared/value_objects/money.py]
7. [domain/shared/value_objects/temporal.py]
8. [domain/shared/value_objects/status.py]
9. [domain/shared/entities.py]
10. [domain/shared/errors.py]
11. [domain/shared/events.py]

Módulo [business/] (completo):

1. [domain/business/__init__.py]
2. [domain/business/entities.py] - Tenant, Business, Location, BusinessSettings
3. [domain/business/value_objects.py] - Address, ContactInfo, OperatingHours
4. [domain/business/ports.py] - Interfaces de repositorio

Módulo [customers/] (completo):

1. [domain/customers/__init__.py]
2. [domain/customers/entities.py] - Customer, ContactPoint, Consent
3. [domain/customers/value_objects.py] - CustomerPreferences, ConsentType
4. [domain/customers/ports.py] - ICustomerRepository

6 módulos esqueleto (2 archivos cada uno = 12): 29-40. [catalog/], [resources/], [availability/], [reservations/], [orders/], [fulfillment/] - cada uno con [entities.py] (placeholder) + [ports.py]

Application, Infrastructure, API, Verticals (marcadores):

1. [application/__init__.py]
2. [application/ports/__init__.py]
3. [application/ports/repositories.py]
4. [infrastructure/__init__.py]
5. [api/__init__.py]
6. [verticals/__init__.py]

Tests (~17 archivos)

1. [tests/__init__.py]
2. [tests/conftest.py]
3. [tests/architecture/__init__.py]
4. [tests/architecture/test_architecture_boundaries.py]

51-56. [tests/unit/domain/shared/] (5 archivos de test + init) 57-60. [tests/unit/domain/business/] (3 archivos de test + init)

61-63. [tests/unit/domain/customers/] (test + 2 inits)

ESTIMACIÓN TOTAL: ~63 archivos nuevos.

D. Responsabilidades de cada módulo

`domain/shared/` - Núcleo agnóstico transversal

Submódulo	Responsabilidad
[value_objects/ids.py]	Tipos fuertes de identidad: [TenantId], [BusinessId], [LocationId], [CustomerId], [EntityId[T]] genérico. Todos incluyen validación. Multi-tenant desde el diseño.
[value_objects/money.py]	[Money] (Decimal + currency code ISO-4217) + [Currency]. INVARIANTE: nunca float. Operaciones: add, subtract, multiply (solo entero), zero. Moneda distinta = error.
[value_objects/temporal.py]	[DateRange] (fecha local inclusiva), [TimeRange] (datetime OBLIGATORIAMENTE timezone-aware). Helpers: [require_aware()], [same_timezone()], [overlap()], [as_utc()]. INVARIANTE: ningún datetime ingenuo (tz=None) cruza el dominio.
[value_objects/status.py]	[LifecycleStatus] enum (DRAFT/ACTIVE/SUSPENDED/CANCELLED/COMPLETED/ARCHIVED). [StatusTransition] (from->to válido + método [ensure()]) que lanza [StatusTransitionError]. Base para máquinas de estado finitas.
[entities.py]	[BaseEntity[T]] (id tipado genérico, created_at, updated_at, igualdad por ID). [AggregateRoot[T]] (extiende BaseEntity + colección de DomainEvent).
[errors.py]	Jerarquía de excepciones: [DomainError] -> [InvariantViolation], [StatusTransitionError], [MoneyCurrencyMismatchError], [TemporalRangeError], [TenantBoundaryViolationError].
[events.py]	[DomainEvent] dataclass base: event_id, occurred_at (UTC), aggregate_id/type, tenant_id/business_id/location_id opcionales, metadata dict.

`domain/business/` - Jerarquía multi-tenant

Archivo	Responsabilidad
[entities.py]	[Tenant] (raíz legal), [Business] (unidad operativa bajo Tenant), [Location] (establecimiento físico/virtual bajo Business), [BusinessSettings] (VO embebido). Cada entidad lleva su cadena de tenancy completa (tenant_id + business_id + location_id cuando corresponde) para aislamiento lógico sin JOINS.
[value_objects.py]	[Address] (línea 1/2, ciudad, región, código postal, país ISO-3166, coordenadas opcionales). [ContactInfo] (teléfono, email, web). [OperatingHours] (7 días, cada uno con TimeRange opcional).
[ports.py]	Protocolos (typing.Protocol) [ITenantRepository] (get, list), [IBusinessRepository] (get, list_by_tenant), [ILocationRepository] (get, list_by_business).

`domain/customers/` - Gestión universal de clientes

Archivo	Responsabilidad
[entities.py]	[Customer] (bajo Tenant/Business/Location), [ContactPoint] (canal + verificación), [Consent] (tipo + fecha + revocable). Customer.contact_points requiere >=1 para estado ACTIVE.
[value_objects.py]	[CustomerPreferences] (idioma, canal preferido, retención). [ConsentType] enum (marketing, profiling, terms, data_sharing).
[ports.py]	[ICustomerRepository] (get, get_by_external_ref, search).

Módulos esqueleto (Catalog, Resources, Availability, Reservations, Orders, Fulfillment)

- Objetivo en 0.1: Establecer topología y contratos, no lógica.
- Cada módulo contiene:
 - [entities.py]: una [@dataclass] placeholder por entidad con los campos mínimos de tenancy ([id], [tenant_id],

[business_id]), y un docstring [TODO: Implementación detallada en FASE 1].

- [ports.py]: [[Modulo]Repository] como Protocol con métodos [get(id)] y [list_by_business(business_id)].

`application/`

Archivo	Responsabilidad
[ports/repositories.py]	Re-exporta (con [__all__]) todos los [I*Repository] de domain como conveniencia. NO introduce lógica. Los casos de uso commands/queries vienen en FASE 1.

`infrastructure/`, `api/`, `verticals/`

- Solo [__init__.py] vacíos. Cualquier import desde domain hacia ellos falla en los tests de arquitectura.

E. Plan de ADRs (ADR-001 a ADR-005)

Formato estándar de cada ADR:

```
# ADR-NNN: <Título>
## Contexto
## Decisión
## Consecuencias
- Positivas:
- Negativas:
- Riesgos y mitigaciones:
```

ADR-001: Monolito modular vs Microservicios

- Contexto: El PDF recomienda monolito modular inicial. ¿Por qué no microservicios desde el día 1?
- Decisión:
 - Adoptar monolito modular con límites estrictos por paquete.
 - Separación fuerte por test de arquitectura (import-guard).
 - Migrar a microservicios SOLO cuando se cumplan TRES condiciones: (a) 2+ verticales en producción, (b) límites de dominio validados empíricamente, (c) necesidad operacional demostrada (escalado independiente / equipos autónomos).
- Consecuencias: Despliegue único -> simplicidad. Riesgo de "monolito distribuido" si los límites se rompen -> mitigado por tests arquitectónicos.

ADR-002: Estrategia multi-tenant y aislamiento

- Contexto: PDF requiere multi-tenant desde el diseño. Jerarquía [Tenant -> Business -> Location].
- Decisión:
 - Aislamiento lógico por columna: TODAS las tablas/entidades operacionales llevan [tenant_id] + [business_id] + [location_id] (cuando aplica).
 - Redundancia intencional: Location lleva [tenant_id] aunque se pueda inferir via Business -> Tenant (justificación: queries sin JOIN, filtros simples, auditoría simple). Invariante: [location.tenant_id == location.business.tenant_id].
 - NO multi-schema ni multi-DB en esta fase.
 - Repositorios validan en cada consulta que no crucen boundaries de tenant.
- Consecuencias: Baja complejidad operativa; filtros obligatorios en queries (probar con tests de isolation).

ADR-003: Modelo Product / Service / CatalogItem

- Contexto: El core debe representar productos físicos, servicios y composiciones sin conceptos específicos.
- Decisión:
 - [CatalogItem] (entidad base) con [type: CatalogItemType] ([PRODUCT], [SERVICE], [BUNDLE], [DIGITAL]).
 - [Variant] = SKU/precio específico por combinación de atributos.
 - [ModifierGroup] / [ModifierOption] = extras configurables.
 - Nombres prohibidos en core: [PiezaDePollo], [ComboPicaPollo], [Yuca], [Platano], [MesaRestaurante]. Se mapean: pica-pollo combos -> [BUNDLE] + [ModifierOption]; peluquería corte -> [SERVICE] + [Variant.duration_minutes].
 - Detalle de implementación: FASE 1. Este ADR fija la estructura.
- Consecuencias: Ambas verticales (pica-pollo / peluquería) se representan sin tocar el core. Riesgo de sobre-abstracción -> mitigado por feature flags en BusinessSettings.

ADR-004: Eventos de dominio y patrón Outbox

- Contexto: PDF requiere desacoplar notificaciones, auditoría e integraciones vía eventos.
- Decisión:
 - [AggregateRoot] colecciona [DomainEvent] lista (no persistida en la entidad misma).
 - FASE 2: Application services persisten aggregate + eventos en tabla [outbox] en la MISMA transacción (consistencia ACID). Un dispatcher separado lee outbox y publica.
 - ENTREGA 0.1: Solo [DomainEvent] base + mecanismo [record_event()/events()/clear_events()] en [AggregateRoot]. Persistencia outbox en FASE 3.
 - Campos obligatorios en todo evento: [event_id] (UUID v4), [occurred_at] (UTC aware), [aggregate_id], [aggregate_type], [tenant_id] (si aplica), [metadata].
- Consecuencias: Consistencia transaccional. Complejidad de entrega -> mitigado por outbox table.

ADR-005: Modelo de extensiones verticales (Regla de Oro)

- Contexto: El pica-pollo NO debe contaminar el core. ¿Cómo se extiende sin romper el core?
- Decisión - 3 niveles, ordenados por preferencia:
 1. Configuración: [BusinessSettings] + feature flags.
 2. Datos semilla: [/verticals/<name>/seeds/] (catalogo, modifier groups, zonas).

- 3. Entidades específicas de vertical (solo si 1 y 2 son insuficientes): [/verticals/<name>/extensions/] con subclases o entidades propias.
 - DIRECCIÓN DEPENDENCIAS (SOLO ESTA PERMITIDA):
[] verticals -> application -> domain
[] [domain] NUNCA importa de [verticals]. [application] NUNCA importa de [verticals]. Se valida con architecture tests.
 - Consecuencias: Nuevos verticales son carpetas nuevas. Riesgo de "feature envy" en el core -> mitigado por el test de nombres prohibidos.
-

F. Diseño de Value Objects

F.1 IDs (`domain/shared/value\_objects/ids.py`)

Decisión de diseño (Riesgo 2 resuelto): Wrapper dataclass inmutable en lugar de NewType, para obtener validación en runtime, no solo en mypy.

```
# Pseudocódigo - estructura (NO es el código final)
from __future__ import annotations
import uuid
from dataclasses import dataclass
from typing import Generic, TypeVar

T = TypeVar("T")

@dataclass(frozen=True)
class EntityId(Generic[T]):
    value: uuid.UUID

    @classmethod
    def new(cls) -> "EntityId[T]":
        return cls(uuid.uuid4())

    def __post_init__(self) -> None:
        if self.value is None or not isinstance(self.value, uuid.UUID):
            raise ValueError("EntityId debe ser un UUID válido")

# Aliases específicos (también wrappers para type safety total)
@dataclass(frozen=True)
class TenantId:
    value: uuid.UUID
    @classmethod
    def new(cls) -> "TenantId": ...
    # misma validación

@dataclass(frozen=True)
class BusinessId: ...      # idéntica estructura
@dataclass(frozen=True)
class LocationId: ...      # idéntica estructura
@dataclass(frozen=True)
class CustomerId: ...      # idéntica estructura
```

Invariantes:

- [value] es siempre un [uuid.UUID] (nunca str, nunca None).
- [TenantId] != [BusinessId] - no son asignables intercambiabilmente (my lo detecta + runtime check si fuera necesario).

F.2 Money (`domain/shared/value\_objects/money.py`)

```
# Pseudocódigo
from __future__ import annotations
from dataclasses import dataclass
from decimal import Decimal
import typing

ISO4217 = typing.Literal["DOP", "USD", "EUR"]  # ampliar según necesidades

@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: ISO4217

    def __post_init__(self) -> None:
        # DEFENSA EN PROFUNDIDAD:
        # (a) float capturado por mypy strict
        # (b) runtime también lo rechaza (puede haber errores de cast dinámico)
        if isinstance(self.amount, float):
            raise TypeError("Money.amount NO acepta float; usa Decimal(str(monto))")
        if not isinstance(self.amount, Decimal):
            raise TypeError("Money.amount debe ser Decimal")
        if self.amount.is_nan() or self.amount.is_infinite():
            raise ValueError("Money.amount no puede ser NaN ni infinito")
        if len(self.currency) != 3 or not self.currency.isalpha():
            raise ValueError(f"Moneda inválida: {self.currency!r}")

    def add(self, other: "Money") -> "Money": ...
    def subtract(self, other: "Money") -> "Money": ...
    def multiply_int(self, factor: int) -> "Money": ...  # solo entero
    @classmethod
    def zero(cls, currency: ISO4217) -> "Money": ...
```

Test clave que DEBE pasar:

- [Money(1.5, "USD")] -> TypeError (por float runtime check)
- [Money(Decimal("1.10"), "USD").add(Money(Decimal("0.20"), "EUR"))] -> [MoneyCurrencyMismatchError]
- [Decimal("0.1") + Decimal("0.2") == Decimal("0.30")] -> exacto

F.3 Temporal (`domain/shared/value\_objects/temporal.py`)

```

# Pseudocódigo
from __future__ import annotations
from dataclasses import dataclass
import datetime as dt

def require_aware(dt_in: dt.datetime) -> dt.datetime:
    """Lanza ValueError si dt_in no tiene zona horaria."""
    if dt_in.tzinfo is None or dt_in.tzinfo.utcoffset(dt_in) is None:
        raise ValueError("El datetime DEBE ser timezone-aware")
    return dt_in

@dataclass(frozen=True)
class DateRange:
    start: dt.date
    end: dt.date
    def __post_init__(self) -> None:
        if self.start > self.end:
            raise TemporalRangeError("DateRange: start > end")

@dataclass(frozen=True)
class TimeRange:
    start: dt.datetime
    end: dt.datetime
    def __post_init__(self) -> None:
        require_aware(self.start)
        require_aware(self.end)
        if self.start.tzinfo != self.end.tzinfo:
            raise TemporalRangeError("TimeRange: ambos extremos deben tener la misma zona horaria")
        if self.start > self.end:
            raise TemporalRangeError("TimeRange: start > end")

def overlap(a: TimeRange, b: TimeRange) -> bool: ...
def as_utc(dt_in: dt.datetime) -> dt.datetime: ...

```

F.4 Status / Lifecycle (`domain/shared/value\_objects/status.py`)

```
# Pseudocódigo
from enum import Enum

class LifecycleStatus(str, Enum):
    DRAFT = "draft"
    ACTIVE = "active"
    SUSPENDED = "suspended"
    CANCELLED = "cancelled"
    COMPLETED = "completed"
    ARCHIVED = "archived"

_LIFECYCLE_TRANSITIONS: dict[LifecycleStatus, set[LifecycleStatus]] = {
    LifecycleStatus.DRAFT: {LifecycleStatus.ACTIVE, LifecycleStatus.CANCELLED},
    LifecycleStatus.ACTIVE: {LifecycleStatus.SUSPENDED, LifecycleStatus.COMPLETED,
                             LifecycleStatus.CANCELLED, LifecycleStatus.ARCHIVED},
    LifecycleStatus.SUSPENDED: {LifecycleStatus.ACTIVE, LifecycleStatus.ARCHIVED},
    LifecycleStatus.CANCELLED: {LifecycleStatus.ARCHIVED},
    LifecycleStatus.COMPLETED: {LifecycleStatus.ARCHIVED},
    LifecycleStatus.ARCHIVED: set(),
}

class StatusTransition:
    @staticmethod
    def can_transition(from_: LifecycleStatus, to: LifecycleStatus) -> bool: ...
    @staticmethod
    def ensure(from_: LifecycleStatus, to: LifecycleStatus) -> None:
        # Lanza StatusTransitionError si inválido
```

G. Diseño de Entidades (core)

G.1 Jerarquía y clases base

```
BaseEntity[T]
??? id: EntityId[T] (o ID específico: TenantId, etc.)
??? created_at: datetime (UTC aware)
??? updated_at: datetime (UTC aware)
??? __eq__ / __hash__ por id

AggregateRoot[T] (extends BaseEntity)
??? _events: list[DomainEvent]
??? record_event(event: DomainEvent) -> None
??? events() -> list[DomainEvent] # copia defensiva
??? clear_events() -> None
```

G.2 Tenant - Agregado raíz de tenancy

Campo	Tipo	Invariantes
[id]	[TenantId]	UUID v4, no nulo
[legal_name]	[str]	2 <= longitud <= 200
[display_name]	[str]	2 <= longitud <= 100
[status]	[LifecycleStatus]	DRAFT -> ACTIVE -> {SUSPENDED / CANCELLED / COMPLETED} -> ARCHIVED
[created_at]	[datetime] (UTC aware)	No nulo
[updated_at]	[datetime] (UTC aware)	No nulo, >= created_at

G.3 Business - Entidad secundaria bajo Tenant

Campo	Tipo	Invariantes
[id]	[BusinessId]	UUID v4
[tenant_id]	[TenantId]	OBLIGATORIO (multi-tenant)
[name]	[str]	2-100 chars
[description]	`str`	None` <= 1000 chars
[legal_info]	[BusinessLegalInfo] (VO)	tax_id (opcional por jurisdicción), legal_name
[contact_info]	[ContactInfo] (VO)	email o phone al menos uno presente
[settings]	[BusinessSettings] (VO)	default_currency: ISO4217, feature_flags: dict[str, bool]
[status]	[LifecycleStatus]	igual máquina que Tenant
[created_at] / [updated_at]	[datetime] UTC aware	-

G.4 Location - Tercer nivel bajo Business

Campo	Tipo	Invariantes	
[id]	[LocationId]	UUID v4	
[tenant_id]	[TenantId]	OBLIGATORIO (redundante pero esencial para isolation sin JOIN)	
[business_id]	[BusinessId]	OBLIGATORIO	
[name]	[str]	2-100 chars	
[address]	[Address] (VO)	país ISO-3166-1 alpha-2 OBLIGATORIO	
[timezone]	[str] (IANA, ej. ["America/Santo_Domingo"])	OBLIGATORIO. Todas las reglas de disponibilidad evalúan en esta tz. Usar [zoneinfo.ZoneInfo] para validar + dependencia condicional [tzdata] en Windows.	
[operating_hours]	[OperatingHours] (VO)	7 días; cada día puede tener 0 o más TimeRange (cerrado si ninguno)	
[contact_info]	`ContactInfo`	None`	Si None -> hereda el de Business
[status]	[LifecycleStatus]	-	
[created_at] / [updated_at]	[datetime] UTC aware	-	
Invariante inter-entidad:	[location.tenant_id == location.business.tenant_id]	Factory constructor valida esto.	

G.5 Customer - Cliente del negocio

Campo	Tipo	Invariantes	
[id]	[CustomerId]	UUID v4	
[tenant_id]	[TenantId]	OBLIGATORIO	
[business_id]	`BusinessId`	None`	None = customer del tenant completo (no asociado a un business concreto)
[location_id]	`LocationId`	None`	None = no asociado a una location concreta
[external_ref]	`str`	None`	ID del sistema de origen (WhatsApp ID, CRM ID, etc.). Búsqueda optimizada.
[given_name]	[str]	1 <= longitud <= 100	
[family_name]	`str`	None`	1 <= longitud <= 100
[full_name]	[property str]	[(given_name + " " + family_name).strip()]	
[contact_points]	[list[ContactPoint]]	len >= 1 para pasar a estado ACTIVE	
[addresses]	[list[Address]]	0..N; un "default" puede marcarse en Address.	
[consents]	[list[Consent]]	GDPR/LOPD-ready. Ningún envío de marketing sin ConsentType.MARKETING=ACTIVE	
[preferences]	[CustomerPreferences] (VO)	idioma (ISO 639-1), canal preferido, retención	
[status]	[LifecycleStatus]	DRAFT hasta que contact_points >=1 -> luego ACTIVE	
[created_at] / [updated_at]	[datetime] UTC aware	-	

G.6 Entidades esqueleto (CatalogItem, Resource, Reservation, Order, Fulfillment)

Ejemplo placeholder para [CatalogItem] (el resto son iguales):

```
# domain/catalog/entities.py
"""
Módulo domain.catalog
TODO: Implementación detallada en FASE 1 del roadmap.
      En esta entrega solo se establece el placeholder con tenancy mínima.
"""
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime
import uuid
from universal_business.domain.shared.value_objects.ids import BusinessId, EntityId, TenantId

CatalogItemId = EntityId["CatalogItem"]

@dataclass
class CatalogItem:  # PLACEHOLDER - no representa la estructura final
    id: CatalogItemId
    tenant_id: TenantId
    business_id: BusinessId
    created_at: datetime
    updated_at: datetime
```

H. Diseño de Puertos / Interfaces de Repositorio

H.1 Principios generales

- [typing.Protocol] (no [abc.ABC]): duck-typing estático, fakes en tests sin herencia. [mypy] valida el contrato.
- Ningún import de SQLAlchemy, FastAPI, bases de datos.
- Toda consulta lleva contexto de tenancy. (No existe método [list()] sin filtro de tenant/business en repos de entidades operacionales.)
- Retornos: entidades de dominio puras. Sin dicts, sin tuplas.
- Semántica > CRUD genérico: nombres como [list_by_tenant()], [get_by_external_ref()], no [find_all()].

H.2 Interfaces por módulo

```
# =====
# domain/business/ports.py
# =====
from typing import Protocol
from universal_business.domain.shared.value_objects.status import LifecycleStatus
from universal_business.domain.business.entities import (
    Tenant, Business, Location, TenantId, BusinessId, LocationId,
)

class ITenantRepository(Protocol):
    def get(self, tenant_id: TenantId) -> Tenant | None: ...
    def list(self, *, status: LifecycleStatus | None = None) -> list[Tenant]: ...

class IBusinessRepository(Protocol):
    def get(self, business_id: BusinessId) -> Business | None: ...
    def list_by_tenant(self, tenant_id: TenantId) -> list[Business]: ...

class ILocationRepository(Protocol):
    def get(self, location_id: LocationId) -> Location | None: ...
    def list_by_business(self, business_id: BusinessId) -> list[Location]: ...

# =====
# domain/customers/ports.py
# =====
class ICustomerRepository(Protocol):
    def get(self, customer_id: CustomerId) -> Customer | None: ...
    def get_by_external_ref(self, *, tenant_id: TenantId,
                           external_ref: str) -> Customer | None: ...
    def search(self, *, tenant_id: TenantId, query: str) -> list[Customer]: ...

# =====
# Otros 6 módulos esqueleto:
#   ICatalogRepository, IResourceRepository, IAvailabilityRepository,
#   IReservationRepository, IOrderRepository, IFulfillmentRepository
# Cada uno con get() y list_by_business().
# =====
```

H.3 Agregado en Application

```
# application/ports/repositories.py
from universal_business.domain.business.ports import (
    ITenantRepository, IBusinessRepository, ILocationRepository,
)
from universal_business.domain.customers.ports import ICustomerRepository
from universal_business.domain.catalog.ports import ICatalogRepository
from universal_business.domain.resources.ports import IResourceRepository
from universal_business.domain.availability.ports import IAvailabilityRepository
from universal_business.domain.reservations.ports import IReservationRepository
from universal_business.domain.orders.ports import IOrderRepository
from universal_business.domain.fulfillment.ports import IFulfillmentRepository

__all__ = [
    "ITenantRepository", "IBusinessRepository", "ILocationRepository",
    "ICustomerRepository", "ICatalogRepository", "IResourceRepository",
    "IAvailabilityRepository", "IReservationRepository",
    "IOrderRepository", "IFulfillmentRepository",
]
```

I. Diseño de Eventos de Dominio base

```

# domain/shared/events.py
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any, ClassVar
import uuid

def _utc_now() -> datetime:
    return datetime.now(tz=timezone.utc)

@dataclass(frozen=True)
class DomainEvent:
    """Base inmutable para todos los eventos de dominio."""

    event_id: uuid.UUID = field(default_factory=uuid.uuid4)
    occurred_at: datetime = field(default_factory=_utc_now)
    aggregate_id: str = field(default_factory=lambda: str(aggregate.id.value)) # str(aggregate.id.value) serializable
    aggregate_type: str = field(default_factory=lambda: "Tenant", "Customer", "Order"...
    tenant_id: str | None = None
    business_id: str | None = None
    location_id: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)

    event_type: ClassVar[str] = "domain.event" # override en subclases

    def __post_init__(self) -> None:
        # occurred_at siempre UTC-aware
        if self.occurred_at.tzinfo is None or self.occurred_at.tzinfo.utcoffset(self.occurred_at) is None:
            raise ValueError("DomainEvent.occurred_at DEBE ser timezone-aware (UTC recomendado)")
        # metadata es un dict copiado defensivamente en subclass init si hace falta

```

Mecanismo en AggregateRoot

```

# domain/shared/entities.py
class AggregateRoot(BaseEntity[T]):
    _events: list[DomainEvent] = field(default_factory=list, init=False)

    def record_event(self, event: DomainEvent) -> None:
        self._events.append(event)

    def events(self) -> list[DomainEvent]:
        return list(self._events) # copia defensiva: el caller no puede mutar la lista real

    def clear_events(self) -> None:
        self._events.clear()

```

Eventos previstos (subclases se implementan en FASE 1, no en 0.1)

- [TenantCreated / TenantStatusChanged]
- [BusinessCreated / BusinessStatusChanged / BusinessSettingsUpdated]
- [LocationCreated / LocationOperatingHoursUpdated]
- [CustomerCreated / CustomerContactPointVerified / CustomerConsentUpdated]

En 0.1 solo se implementa la base genérica ([DomainEvent] + mecanismo en [AggregateRoot]).

J. Estrategia de Tests

J.1 Distribución (pirámide para 0.1)

```
????????????????????????????????????????????????????????
? 2 tests de arquitectura (import boundaries) ? <- 0.1
????????????????????????????????????????????????????????
? ~35 tests unitarios de dominio ? <- 0.1
? (shared ~22 + business ~8 + customers ~5) ?
????????????????????????????????????????????????????????
? 0 application / integration tests ? (FASE 1/2)
????????????????????????????????????????????????????????
```

J.2 Tests de Arquitectura (`tests/architecture/test\_architecture\_boundaries.py`)

Implementación: inspección [importlib] + [sys.modules] (sin librerías adicionales en 0.1; evaluar [import-linter] para FASE 1).

#	Prueba	Cómo
AT-1	[domain] NO importa de [infrastructure], [api], [verticals].	Importar cada submódulo de [domain]; inspeccionar [sys.modules] por nombres prohibidos.
AT-2	[domain.shared] NO importa de módulos específicos (business, customers, catalog...).	AST parse de cada archivo en [domain/shared/].
AT-3	[application] NO importa de [infrastructure], [api], [verticals].	Análogo a AT-1.

AT-4	[domain] NO contiene paquetes excluidos en sys.modules: [fastapi], [sqlalchemy], [psycopg], [whatsapp], [firebase], [react].	[import domain], comprobar [k for k in sys.modules if <pattern>].
AT-5	Ningún nombre de pica-pollo filtrado aparece en names/symbols/comentarios de [domain/] y [application/]: [restaurant], [piccapollo], [pica.*pollo], [pollo], [yuca], [platano], [mesarestaurante], [repartidorrestaurant], [salon], [clinic].	[grep] con flag case-insensitive sobre los [.py] (con [re.IGNORECASE]).
AT-6	Los 6 módulos esqueleto importan sin levantar DB ni dependencias externas.	Importar cada uno en aislamiento; capturar ImportError / ModuleNotFound.

J.3 Tests unitarios de Value Objects

[test_ids.py] (~=6 tests):

1. [TenantId.new()] genera UUID v4 válido.
2. [TenantId(uuid_v4_string_inválido)] -> ValueError.
3. [TenantId == CustomerId] con mismo UUID interno -> False (distinto tipo).
4. Igualdad y hash: [TenantId(X) == TenantId(X)] y hash igual.
5. Representación str/repr útil.
6. [EntityId[T]] new/validación genérica.

[test_money.py] (~=12 tests): 1-2. [Money(float, "USD")] -> TypeError; [Money(int, "USD")] -> TypeError o coerce explícito? (Decisión: error para int también, usar [Decimal(int)] si hace falta).

1. [Decimal("0.1") + Decimal("0.2") == Decimal("0.30")] exacto.
2. [add()] misma moneda OK; operación conmutativa.
3. [add()] moneda distinta -> [MoneyCurrencyMismatchError].
4. [subtract()] igual moneda OK; resultado negativo permitido (indica deuda/descuento).
5. [multiply_int(3)] OK; [multiply_int(1.5)] -> TypeError.
6. [zero("DOP")] es [Money(Decimal("0.00"), "DOP")].
7. Comparaciones [==], [<], [>], [<=], [>=] (solo misma moneda, si no -> error).
8. [amount] NaN/infinito -> ValueError.
9. Moneda inválida ["PE"] (longitud !=3) -> ValueError.
10. Moneda ["dop"] minúscula -> aceptada/normalizada? (Decisión: str.upper en [__post_init__]).

[test_temporal.py] (~=10 tests):

1. [require_aware(datetime_ingenuo)] -> ValueError.
2. [require_aware(datetime_utc)] devuelve ok.
3. [DateRange(ayer, hoy)] OK; [DateRange(hoy, ayer)] -> [TemporalRangeError].
4. [TimeRange] con ambos extremos misma tz OK.
5. [TimeRange] con extremos en distintas tz -> error.
6. [TimeRange] start > end -> error.
7. [overlap(r1, r2)]: r1 anterior, r2 posterior -> False.
8. [overlap(r1, r2)]: intersección estricta -> True.
9. [overlap(r1, r2)]: touching (r1.end == r2.start) -> False (contigüidad no es solapamiento).
10. [as_utc(dt_con_tz_distinta)] devuelve UTC equivalente.

[test_status.py] (~=5 tests):

1. DRAFT -> ACTIVE permitido; [ensure()] no lanza.
2. ACTIVE -> DRAFT prohibido; [ensure()] lanza [StatusTransitionError].
3. ARCHIVED -> nada permitido (estado terminal).
4. [can_transition()] retorna bool correcto para todos los casos de la tabla.
5. Subclase (ej. [OrderStatus] si hubiera) puede extender máquina sin romper base.

[test_domain_events.py] (~=4 tests):

1. [DomainEvent] sin tz -> ValueError en post_init.
2. [DomainEvent] con UTC -> [occurred_at.tzinfo] presente.
3. [aggregate.record_event(e)] -> [aggregate.events()] retorna lista con [e].
4. Modificar la lista retornada por [events()] NO afecta la interna del aggregate (copia defensiva).
5. [clear_events()] -> [events()] retorna lista vacía.

J.4 Tests unitarios de Entidades

[test_tenant.py] (~=4):

1. Constructor happy path crea instancia válida.
2. Falta [legal_name] / [display_name] -> ValueError.
3. Status DRAFT -> ACTIVE permitido; ACTIVE -> DRAFT error.
4. Igualdad por ID: 2 tenants mismo ID, datos distintos -> iguales.

[test_business.py] (~=4):

1. Constructor happy path con settings.
2. [contact_info] sin email ni phone -> [InvariantViolation].
3. [settings.default_currency] inválido -> error en VO [BusinessSettings].
4. Status transitions.

[test_location.py] (~=5):

1. Constructor happy path con timezone IANA válida.
2. Timezone ["Foo/Bar"] inválido -> ValueError (usa [zoneinfo] validation).
3. [address.country] no ISO-2 -> error en [Address].
4. Invariante [tenant_id == business.tenant_id]: factory valida, falla si no coincide.
5. [contact_info is None] -> property calculada retorna [business.contact_info].

[test_customer.py] (~=5):

1. Constructor happy path con 1 contact_point.
2. Transición DRAFT -> ACTIVE con contact_points vacíos -> error.

3. [full_name] con solo given_name -> sin espacios sobrantes.
4. [search_consent(type=MARKETING)] retorna solo esos (si existe método en Customer).
5. Igualdad por CustomerId.

J.5 Fixtures (tests/conftest.py)

```

"""Fixtures base reutilizables por TODOS los tests."""
import pytest
from datetime import timezone
from universal_business.domain.shared.value_objects.ids import (
    TenantId, BusinessId, LocationId, CustomerId,
)

TENANT_ID = TenantId(uuid.UUID("11111111-1111-4111-8111-111111111111"))
BUSINESS_ID = BusinessId(uuid.UUID("22222222-2222-4222-8222-222222222222"))
LOCATION_ID = LocationId(uuid.UUID("33333333-3333-4333-8333-333333333333"))
CUSTOMER_ID = CustomerId(uuid.UUID("44444444-4444-4444-8444-444444444444"))

TZ_SANTO_DOMINGO = "America/Santo_Domingo"
DEFAULT_CURRENCY = "DOP"

@pytest.fixture
def utc_now():
    return datetime.now(tz=timezone.utc)

# + fixtures: sample_tenant() / sample_business() / sample_location() / sample_customer()

```

J.6 Coverage target (mínimo 0.1)

Módulo	Mínimo
[domain/shared/]	>= 90%
[domain/business/]	>= 80%
[domain/customers/]	>= 70%
Global	>= 60% (por los esqueletos vacíos)

K. Estrategia de Tipado Estático / Linting

K.1 Stack de herramientas (en pyproject.toml)

Categoría	Herramienta	Versión mín	Justificación
Linting + isort + format	ruff	>= 0.6	Todo en uno; veloz; reemplaza flake8+isort+black
Type checking	mypy	>= 1.10	Estándar de facto; strict mode
Test runner	pytest	>= 8.0	Standard
Coverage	pytest-cov	>= 5.0	Coverage integrado
Seguridad (AST)	bandit	>= 1.7	Detección de antipatronos de seguridad

K.2 Dependencias de proyecto (opcional-dependencies)

```
# En pyproject.toml:
[project]
name = "universal-business-core"
version = "0.1.0"
description = "Universal Business Core - dominio agnóstico multi-tenant"
requires-python = ">=3.11"
dependencies = [
    "tzdata; sys_platform == 'win32'",    # Windows no lleva tzdata en stdlib
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-cov>=5.0",
    "ruff>=0.6",
    "mypy>=1.10",
    "bandit>=1.7",
]
test = ["pytest>=8.0", "pytest-cov>=5.0"]
lint = ["ruff>=0.6", "mypy>=1.10", "bandit>=1.7"]
```

K.3 Configuración Mypy (progresiva)

```
[tool.mypy]
python_version = "3.11"
strict = true
warn_return_any = true
warn_unused_configs = true
explicit_package_bases = true
ignore_missing_imports = false
show_error_codes = true
my_path = "src:tests"

# Relajación TEMPORAL para los 6 módulos esqueleto:
[[tool.mypy.overrides]]
module = [
    "universal_business.domain.catalog.*",
    "universal_business.domain.resources.*",
    "universal_business.domain.availability.*",
    "universal_business.domain.reservations.*",
    "universal_business.domain.orders.*",
    "universal_business.domain.fulfillment.*",
]
strict = false
```

K.4 Configuración ruff

```
[tool.ruff]
target-version = "py311"
line-length = 99
src = ["src", "tests"]
exclude = [".venv", "build", "dist", "docs"]

[tool.ruff.lint]
# Ruleset recomendado: E (pycodestyle) + F (pyflakes) + I (isort) + N (pep8-naming) + UP (pyupgrade)
# + B (flake8-bugbear)
select = ["E", "F", "I", "N", "UP", "B"]
ignore = ["B008"] # ignorar "do not perform function calls in argument defaults" (dataclass lo requiere)

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
```

K.5 Pre-commit (`.pre-commit-config.yaml`)

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.0
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format
- repo: https://github.com/PyCQA/bandit
  rev: 1.7.9
  hooks:
    - id: bandit
      args: ["-r", "src", "--severity-level", "medium"]
```

K.6 Comandos locales

```
# Lint + format
ruff check .
ruff format .

# Type check
mypy src tests

# Tests + coverage
pytest -q --cov=src/universal_business --cov-report=term --cov-fail-under=60

# Seguridad
bandit -r src
```

L. Estrategia CI (`.github/workflows/ci.yml`)

```

name: CI - Architectural Baseline (0.1)

on:
  push:
    branches: [ feat/architectural-baseline, master ]
  pull_request:
    branches: [ '**' ]

jobs:
  quality:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      python-version: ["3.11"]

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev,test,lint]"

      - name: Ruff lint
        run: ruff check . --exit-zero      # temporal; luego --exit-non-zero-on-fix

      - name: Ruff format check
        run: ruff format --check .

      - name: Bandit security
        run: bandit -r src/ -ll || true    # non-blocking en 0.1; estricto FASE 2

      - name: Mypy strict (shared + business + customers)
        run: |
          mypy --strict \
            src/universal_business/domain/shared \
            src/universal_business/domain/business \
            src/universal_business/domain/customers \
            tests

      - name: Pytest + coverage
        run: |
          pytest -q \
            --cov=src/universal_business \
            --cov-report=term \
            --cov-report=xml \
            --cov-report=html \

```

--cov-fail-under=60

- name: Upload coverage artifact
 - uses: actions/upload-artifact@v4
 - with:
 - name: coverage-report
 - path: htmlcov/
-

M. Criterios de Aceptación - GATE 0.1

> "La entrega se acepta solo si el core puede importarse y probarse sin levantar FastAPI, una base de datos o servicios externos, y si no contiene nombres o reglas específicas del pica pollo."

GATE 0.1 - Checklist (15 criterios cuantificables)

ID	Criterio	Método de verificación
G1	[import universal_business.domain] NO requiere paquetes excluidos (FastAPI, SQLAlchemy, psycopg, Firebase, WhatsApp, React).	[tests/architecture/test_architecture_ boundaries.py] -> AT-4
G2	100% tests pasan (~=50-60 tests estimados).	[pytest -q] -> exit code 0
G3	Coverage global >= 60%.	[--cov-fail-under=60]
G4	[ruff check .] -> 0 errores, 0 warnings no excluidos explícitamente.	CI step Ruff lint
G5	[ruff format --check .] -> pasa.	CI step Ruff format
G6	[mypy --strict] pasa sobre [shared/], [business/], [customers/].	CI step Mypy
G7	Ninguna entidad, variable, función, módulo ni docstring en [domain/] + [application/] menciona términos de pica-pollo (ver AT-5).	AT-5 (grep con patrones + case-insensitive)
G8	[Money] no acepta [float] en su API pública (runtime error + mypy strict error).	[test_money.py] - Test 1-2
G9	Ningún constructor de entidad acepta [datetime] sin zona horaria.	[test_temporal.py] + tests de entidad (contact_info con datetime naïve)
G10	[Tenant], [Business], [Location], [Customer] se instancian, comparan y transicionan sin repositorio ni dependencia externa.	Tests de unidad de cada módulo
G11	Interfaces de repositorio son [Protocol] o [ABC]; NO dependen de SQLAlchemy.	Grep for [from sqlalchemy] en domain/ + AT-1
G12	5 ADRs completos en formato estándar.	Revisión humana (checklist PR)

G13	[ARCHITECTURE.md] explica las 4 capas, modelo de tenancy, y estrategia de extensiones verticales.	Revisión humana
G14	Pipeline CI pasa en [ubuntu-latest] + Python 3.11.	GitHub Actions ? green
G15	Los 15 items del scope de la tarea están todos implementados (ver lista SCOPE a continuación).	Revisión humana del PR

SCOPE checklist integrado (15 items)

- 1. ? Skeleton Python del proyecto
- 2. ? [pyproject.toml]
- 3. ? [ARCHITECTURE.md]
- 4. ? ADR-001 a ADR-005
- 5. ? Value objects básicos (IDs, Money, DateRange/TimeRange, status)
- 6. ? Tenant, Business, Location, Customer
- 7. ? Repository interfaces / ports
- 8. ? Skeleton: Catalog, Resource, Availability, Reservation, Order, Fulfillment
- 9. ? Domain events base
- 10.? Architecture tests
- 11.? Unit-test structure
- 12.? pytest
- 13.? Linting (ruff)
- 14.? Static type checking (mypy strict)
- 15.? Minimal CI configuration

N. Riesgos y Ambigüedades detectadas

ID	Riesgo / Ambigüedad	Descripción	Impacto	Mitigación propuesta
R1	NewType vs Wrapper-class para IDs	NewType no aporta seguridad runtime.	Medio	Decisión: Wrapper dataclass ([@dataclass(frozen=True)]) con validación (__post_init__). Queda documentado en ADR-002.
R2	[Currency]: Enum cerrado vs String literal	180 monedas ISO; Enum = mantenimiento; Str sin validar = runtime bugs.	Bajo	[typing.Literal["DOP", "USD", "EUR"]] en 0.1. Expandir la lista en FASE 1. Post_init valida long-3 + alpha.
R3	ABC vs Protocol para repos	ABC fuerza herencia; Protocol = duck typing estático.	Bajo	Protocol (mejor para fakes en tests sin importar).
R4	Location [tenant_id] redundante	PDF no especifica si debe repetirse.	Bajo	Sí se repite. Justificación: aislamiento sin JOIN, auditoría simple. Validado por factory (invariante). ADR-002.
R5	Alcance VO's BusinessSettings / Consent	"lifecycle/status primitives where appropriate" - ambiguo.	Bajo	Minimalismo: solo [LifecycleStatus] shared + [ConsentType] enum. OrderStatus/ReservationStatus son placeholders en esqueletos.
R6	[import-linter] vs tests arquitectónicos caseros	Librería adicional vs. fragilidad de parse AST/importlib.	Bajo	Caseros en 0.1 (inspección sys.modules + grep). Evaluar [import-linter] FASE 1 si hay falsos positivos.

R7	Money strict + float defence	mypy strict detecta float en API. ¿Y si alguien hace [Money(Decimal(str(my_float)))]?	Bajo	Defensa en profundidad: (a) mypy strict; (b) [__post_init__] runtime TypeError si [isinstance(self.amount, float)]; (c) comentario en docstring del constructor.
R8	tzdata en Windows	[zoneinfo.ZoneInfo("America/Santo_Domingo")] falla en Windows sin [tzdata].	Bajo	Dependencia condicional en pyproject: ["tzdata; sys_platform == 'win32'"].
R9	Scope creep 6 módulos esqueleto	Tentación de implementar Order/Reservation ya.	Medio	Regla estricta de esqueleto: máximo [id], [tenant_id], [business_id], [created_at], [updated_at] + TODO docstring. Sin métodos sin invariantes. Aplicar Architecture test que verifique que esas entidades no tengan >5 atributos.
R10	Feature flags en BusinessSettings	¿Qué flags incluir en 0.1? ¿Nombres demasiado verticales?	Bajo	Solo dos flags genéricos: ["has_reservations": bool], ["has_delivery": bool]. Nada de ["has_pica_pollo_combo"].

R11	[LifecycleStatus] demasiado rígido	Order y Reservation tienen ciclos muy distintos al Tenant.	Bajo	[LifecycleStatus] es un enum shared genérico de "estados de vida". Cada módulo define SU PROPIO enum ([OrderStatus(str, Enum)]) con su propia matriz de transiciones. Comparten el helper [StatusTransition].
R12	Windows path + test grep	Test AT-5 usa [pathlib.Path.glob], los paths pueden ser [\\] en Windows.	Bajo	Usar siempre [pathlib.Path] (funciona cross-platform) y leer archivos con [encoding="utf-8"]. CI corre en Ubuntu (paths POSIX).

FIN DEL DOCUMENTO - Plan Entrega 0.1 Architectural Baseline v1.0

Espera la aprobación del usuario antes de proceder a la implementación.